Mathematical Problem-Solving Workflow

Full-fidelity process protocol for `pid-rs`

pid-rs project analysis

24 July 2026

Abstract

This document is the human-readable typeset rendition of `MATHEMATICAL_PROBLEM_SOLVING_WORKFLOW.md`. It records external process observations, then defines the complete project protocol for exact claims, independent routes, retained failures, certificate conversion, adversarial audit, blind holdouts, PID-specific exceptional cases, layered assurance, and claim disposition. The protocol is project-defined guidance. It is not a PID estimator, theorem, benchmark result, or validation claim.

Contents

1 Reader's primer: objects, logic, and evidence boundaries **3**
- 1.1 Status of this primer . . . 3
- 1.2 Four objects that must not be conflated . . . 3
- 1.3 Exact claims, quantifiers, and counterexamples . . . 3
- 1.4 Certificates and independently checkable computation . . . 4
- 1.5 PID vocabulary and a finite worked reconstruction . . . 4
- 1.6 Negative example: an unseen rare cell . . . 5
- 1.7 Negative example: pairwise is not mutual independence . . . 5
- 1.8 Bounded evolutionary search: useful scope and hard boundary . . . 5

2 Typed assurance and rigorous evidence aggregation **5**
- 2.1 Status of this supplement . . . 5
- 2.2 Preserve a vector across assurance layers . . . 5
- 2.3 Critical cut sets determine dependence . . . 6
- 2.4 Worked example: five correlated test families are one confirmation at a shared oracle . . . 6
- 2.5 When a model-conditional posterior belief is permitted . . . 7

3 Mathematical problem-solving workflow for pid-rs **9**
- 3.1 Status and scope . . . 9
- 3.2 Compact glossary . . . 9
- 3.3 Source observations . . . 10
  - 3.3.1 X thread . . . 10
  - 3.3.2 Cycle Double Cover prompt . . . 11
  - 3.3.3 Erdős repository . . . 11
  - 3.3.4 Erdős 477 cubic repository . . . 12
  - 3.3.5 Corrected vector-bundle claim: a typed citation-edge failure . . . 12
  - 3.3.6 External AI discussions . . . 15
- 3.4 AI model operating protocol . . . 15
  - 3.4.1 Applicability and risk . . . 15
  - 3.4.2 Separate roles . . . 15
  - 3.4.3 Independent routes . . . 16
  - 3.4.4 Critical-cut-set accounting . . . 17
  - 3.4.5 Frozen run context . . . 17
  - 3.4.6 Evidence labels and route memos . . . 18
- 3.5 pid-rs protocol . . . 18

3.5.1 1. Write an exact-claim packet . . . 18
3.5.2 2. Build an obligation graph . . . 19
3.5.3 3. Keep an independent approach registry . . . 20
3.5.4 4. Retain blockers and failed routes . . . 20
3.5.5 Load-bearing correction ledger . . . 20
3.5.6 5. Convert computation into certificates . . . 21
3.5.7 6. Run an adversarial audit . . . 21
3.5.7.1 SxPID definition-compatibility firewall . . . 22
3.5.7.2 Imported-theorem application map . . . 22
3.5.7.3 Citation-edge type check . . . 23
3.5.7.4 Cheap invariant probes . . . 24
3.5.8 7. Use a blind holdout protocol . . . 24
3.5.8.1 Minimum blind-benchmark commitment . . . 25
3.5.9 8. Maintain a claim-to-evidence matrix . . . 26
3.6 Application to shared-exclusions PID . . . 27
3.6.1 Exact claim packet for a new PID theorem . . . 27
3.6.2 Independent PID approach families . . . 28
3.6.3 Required counterexample search . . . 28
3.6.4 Formal and software evidence . . . 29
3.6.5 Statistical and ecosystem evidence . . . 29
3.7 Full semantic closure . . . 29
3.8 PID exceptional-case checklist . . . 30
3.9 Layered assurance and go/no-go gates . . . 30
3.10 Acceptance rule . . . 31

1 Reader's primer: objects, logic, and evidence boundaries

1.1 Status of this primer

This section is a typeset-only explanatory expansion. It introduces the vocabulary and works through examples needed to apply the protocol without assuming prior knowledge of formal methods, statistics, or partial information decomposition (PID). It does not change the canonical protocol reproduced in full beginning with Section 3. Every example below is project-defined process guidance, not a new PID theorem or estimator result.

1.2 Four objects that must not be conflated

Let P denote a population law and let $F(P)$ denote a mathematical information functional. Given rows $Z_1, \ldots, Z_n$, an estimator is an algorithmic rule

$$\widehat{F}_n = A_n(Z_1, \ldots, Z_n).$$

A particular program returns a represented number

$$\widetilde{F}_n = \mathrm{Program}_{\mathrm{compiler,features,platform}}(Z_1, \ldots, Z_n),$$

and a consumer may use that number, diagnostics U_n, and a policy $\mathcal{D}$ to make a decision

$$d_n = \mathcal{D}(\widetilde{F}_n, U_n).$$

These are four different objects: the population functional, the estimator, the executable result, and the consumer decision. A proof that F is continuous establishes no finite-sample error rate for $\widehat{F}_n$. An estimator theorem does not prove that the executable implements the estimator. Executable conformance does not establish that a downstream decision is safe. The assurance path therefore has distinct implications:

$$\boxed{\text{definition}} \longrightarrow \boxed{\text{estimator theorem}} \longrightarrow \boxed{\text{executable conformance}} \longrightarrow \boxed{\text{consumer qualification}}.$$

Each arrow is a separate obligation. Evidence for a box cannot silently replace evidence for an arrow.

1.3 Exact claims, quantifiers, and counterexamples

An exact mathematical claim should be reducible to a proposition such as

$$\forall x \in \mathcal{X}, \qquad A_1(x) \wedge \cdots \wedge A_k(x) \Longrightarrow C(x), \tag{1}$$

where $\mathcal{X}$ is the domain, A_i are assumptions, and C is the conclusion. A counterexample is one explicit $x_\star \in \mathcal{X}$ for which every $A_i(x_\star)$ is true but $C(x_\star)$ is false. An example outside $\mathcal{X}$, or one that violates an assumption, does not refute (1); it may instead show that the assumption is necessary.

Quantifier order and convergence mode are part of the claim. For example, pointwise convergence in probability over a declared class $\mathcal{P}$ has the pattern

$$\forall P \in \mathcal{P}\ \forall \varepsilon, \eta > 0\ \exists N(P, \varepsilon, \eta)\ \forall n \geq N(P, \varepsilon, \eta) : \Pr_P\left\{\left|\widehat{F}_n - F(P)\right| > \varepsilon\right\} < \eta. \tag{2}$$

A uniform-in-P statement would require an $N(\varepsilon, \eta)$ that works for every law in $\mathcal{P}$. Equation (2) does not imply that stronger statement. Likewise, $\forall \varepsilon\ \exists N$ cannot be replaced with $\exists N\ \forall \varepsilon$. The claim packet records this order before proof search so that a correct proof of a weaker rearrangement is not mistaken for completion.

An obligation structure is an AND/OR directed acyclic hypergraph. Its nodes are propositions or finite checks. An AND-hyperedge $\{u_1, \ldots, u_r\} \to v$ records that all named premises are needed to derive v; alternative incoming hyperedges are OR-routes, any one of which may derive v. A leaf closes only through accepted evidence of the declared class. An internal node closes only when every tail node of at least one accepted incoming hyperedge is closed and the edge's implication has itself been justified. The packet retains every minimal shared cut set: each smallest set of open or common-cause nodes that intersects every currently accepted route to the target. Rephrasing an open node, or closing one branch of an AND-edge, is not closure.

1.4 Certificates and independently checkable computation

Exploratory floating-point output can suggest a lemma but is not a proof. A certificate separates generation from checking:

$$\text{instance} \xrightarrow{\text{generator}} \text{certificate} \xrightarrow{\text{independent checker}} \{\text{accept}, \text{reject}\}.$$

The checker must state the finite proposition that acceptance establishes. For rational inputs, integer and rational identities should remain exact. Transcendental terms require either a symbolic identity or outward-rounded intervals. If $L \le x \le U$ and a sign is needed, then

$$L > 0 \Longrightarrow x > 0, \qquad U < 0 \Longrightarrow x < 0, \qquad 0 \in [L, U] \Longrightarrow \text{unresolved}.$$

The last branch is not a failure; it is the correct abstention. A useful mutation test changes a sign, endpoint, mask, or denominator in a copy of the evidence. The checker must reject the mutant for the intended reason.

1.5 PID vocabulary and a finite worked reconstruction

PID studies how several sources $S_1, \ldots, S_m$ provide information about a target T. For finite variables, a joint law P assigns nonnegative masses summing to one. A source collection is a nonempty subset of $\{1, \ldots, m\}$. An antichain is a nonempty set of source collections in which no member contains another. PID cumulative quantities c_α, indexed by antichains, are related to atoms π_β by a finite zeta transform

$$c_\alpha = \sum_{\beta \preceq \alpha} \pi_\beta, \qquad \pi_\alpha = \sum_{\beta \preceq \alpha} \mu(\beta, \alpha) c_\beta, \tag{3}$$

where $\preceq$ is the declared redundancy order and μ is its Möbius function. Equation (3) is finite algebra. It does not by itself prove that the cumulative values have the intended paper semantics or that floating-point code evaluates them accurately.

For the ordinary two-source four-atom shape, let R, U_1, U_2, and S denote redundancy, source-specific unique terms, and synergy. If the corresponding cumulative values are c_R, c_1, c_2, c_{12}, inversion is

$$R = c_R, \tag{1}$$
$$U_1 = c_1 - R, \tag{2}$$
$$U_2 = c_2 - R, \tag{3}$$
$$S = c_{12} - R - U_1 - U_2. \tag{4}$$

Substitution gives the reconstruction identities

$$c_1 = R + U_1, \qquad c_2 = R + U_2, \qquad c_{12} = R + U_1 + U_2 + S. \tag{4}$$

An exact checker can verify (4) for every declared finite count table. That closes the finite reconstruction obligation only. Negative signed shared-exclusions atoms are permitted by the functional and must not be clamped merely to make a table look nonnegative.

1.6 Negative example: an unseen rare cell

Let a finite population contain a cell $z_\star$ with mass $P(z_\star) = \rho > 0$. Under independent sampling, the exact probability that $z_\star$ is absent from n rows is

$$\Pr\left\{\widehat{P}_n(z_\star) = 0\right\} = (1-\rho)^n. \tag{5}$$

With $\rho = 10^{-4}$ and $n = 1000$, this probability is approximately 0.9048. Thus an observed support can omit a genuine population cell with high probability. Replacing an unknown population mass floor with the smallest observed mass is therefore invalid without another assumption or confidence construction. Fixed-support continuity remains useful, but it is not a support-discovery theorem or a practical finite-sample certificate.

1.7 Negative example: pairwise is not mutual independence

Let X and Y be independent fair bits and set $Z = X \operatorname{xor} Y$. Every pair among X, Y, Z is independent, but the triple is not: Z is determined by X, Y. Therefore pairwise independence, zero covariance, or a benign autocorrelation plot cannot establish mutual independence of complete rows within a dependence color. A concentration result whose premise is mutual within-color independence needs that premise justified by design or a separate theorem.

1.8 Bounded evolutionary search: useful scope and hard boundary

Genetic or evolutionary search can be useful when fitness is mechanically checkable. For a mutation m of a proof script, certificate, or implementation, a search may rank

$$f(m) = \big(\mathbf{1}\{\text{invalid mutant is accepted}\}, -\text{witness size}, -\text{runtime}\big)$$

lexicographically. Mutation operators may swap a source mask, replace a union with an intersection, reverse a log ratio, perturb an interval endpoint, or delete one Möbius predecessor. Any accepted invalid mutant is a concrete checker weakness and must be retained and minimized.

Failure to find such a mutant is not a proof. An exhaustive finite mutation domain $\mathcal{M}_B$ can support only the bounded statement

$$\forall m \in \mathcal{M}_B, \qquad \mathrm{Checker}(m) = \mathrm{reject}\,.$$

A heuristic genetic run samples only part of its search space. Its valid output is a discovered counterexample or a diagnostic search log, never a theorem-vote or a confidence score.

2 Typed assurance and rigorous evidence aggregation

2.1 Status of this supplement

This is a typeset-only explanatory supplement to the canonical protocol in Section 3. It specifies how evidence records may be aggregated without turning heterogeneous proof, numerical, software, statistical, and consumer evidence into an unsupported score. It is process guidance, not a calibrated confidence model for `pid-rs`.

2.2 Preserve a vector across assurance layers

Evidence has different semantics at different layers. Preserve a typed vector

$$\mathcal{A}(H) = \big(E_{\mathrm{sem}}, E_{\mathrm{math}}, E_{\mathrm{formal}}, E_{\mathrm{num}}, E_{\mathrm{exec}}, E_{\mathrm{stat}}, E_{\mathrm{consumer}}, E_{\mathrm{archive}}\big) \tag{6}$$

for a fully written proposition H. Each component is itself a record

$$E_\ell = (\text{state}, \text{scope}, \text{assumptions}, \text{artifacts}, \text{negative evidence}, \text{residual boundary}). \tag{7}$$

The component *artifact-verification state* is one of `not-applicable`, `open`, `rejected`, or `closed`. It is distinct from the accepted evidence class (for example, theorem, finite machine check, certified bound, empirical result, counterexample, or diagnostic) and from the claim disposition (`proposed`, `active`, `blocked`, `falsified`, or `complete`). Multiple evidence records may support one component; one claim has one current disposition.

The vector and gate map is:

Component	Gate(s)	Required subject
E_{sem}	G0–G1	claim identity, conventions, and premises
E_{math}	G2	mathematical proof or counterexample
E_{formal}	G3	proof-checker semantics and implication
E_{num}	G4	rigorous numerical enclosure
E_{exec}	G5	executable conformance
E_{stat}	G6	scoped estimator calibration
E_{consumer}	G7	consumer contract and qualification
E_{archive}	G8	reproducible evidence and release archive

A machine-checked lattice identity and an open calibration study are not numbers on one common scale. Do not map their labels to integers and average them. An average would erase the fact that one load-bearing layer is open. The decision rule is instead proposition- and layer-specific: every applicable required component must meet its stated closure condition, and every inapplicable component needs a recorded reason.

2.3 Critical cut sets determine dependence

Let test families $T_1, \dots, T_k$ bear on a binary proposition H, and let C range over a fully defined finite set of latent shared-failure states for an oracle, imported theorem, generator, or implementation. Different test names do not make them independent. For each $B \in \{H, \neg H\}$, a complete joint evidence model must retain the shared state:

$$\Pr(T_1, \dots, T_k \mid B) = \sum_c \Pr(C = c \mid B)\ \Pr(T_1, \dots, T_k \mid B, C = c). \tag{8}$$

Multiplying marginal pass probabilities is justified only after an evidence-dependence model establishes the required conditional independence under both H and $\neg H$. A critical-cut-set memo names each shared node, which routes use it, and which independent evidence—if any—closes it. If the latent failure-state space omits an identified common cause, the numerical model is incomplete and the scalar route must abstain.

2.4 Worked example: five correlated test families are one confirmation at a shared oracle

Suppose five suites exercise row permutations, source permutations, small count tables, canonical logic gates, and randomized tables. All five compare the Rust result with the same reference oracle. Let D mean that the implementation has a particular semantic defect, and let O mean that the shared oracle independently contains the same defect. Conditional on $D \wedge O$, every suite can pass:

$$\Pr(T_1 = \cdots = T_5 = \text{pass} \mid D, O) = 1. \tag{9}$$

They are useful for code paths and transformations outside the shared defect, but at the oracle cut set they are one correlated route.

For scale only, consider a hypothetical, fully specified model in which $\Pr(O \mid D) = q = 0.01$, while in the absence of O each suite independently misses D with probability $\varepsilon = 0.01$. Then

$$\Pr(\text{all five pass} \mid D) = q + (1 - q)\varepsilon^5 \approx 0.010000000099, \tag{10}$$

not $\varepsilon^5 = 10^{-10}$. Ignoring the shared oracle would overstate the strength against this defect by about 10^8. The numbers in (10) are an illustration, not calibrated `pid-rs` probabilities. If q and the conditional miss rates have not been externally calibrated, no numerical confidence follows.

The retained raw evidence should say, for example:

Evidence component	Established locally	Shared boundary retained
E_{exec}	Five transformation and corpus families pass	All use oracle C
E_{num}	Oracle values reproduce their own fixtures	No independent enclosure yet
E_{formal}	Finite reconstruction lemma closes	Does not identify oracle semantics
Independent route	Open until a separately written exact or interval checker passes	Must not import C's result table

This vector remains in the evidence packet even if a permitted scalar is later reported.

2.5 When a model-conditional posterior belief is permitted

A scalar is prohibited for a vague project, method, or release. It is permitted only for one fully specified empirical decision proposition H with a truth-labelled external reference class. It is not an operational replacement for deductive truth of a fixed theorem. The packet must include all of the following:

1. an explicit prior $\pi = \Pr(H)$, with provenance and interpretation;
2. calibrated likelihoods $L_\theta(E \mid H)$ and $L_\theta(E \mid \neg H)$;
3. an evidence-dependence model, including every identified shared cut-set node;
4. truth-labelled external calibration data not reused to construct, select, stop, or tune the evaluated evidence;
5. a declared reference class, exchangeability or transport assumption, domain-shift model, and the complete frozen evidence-selection and stopping protocol;
6. finite-calibration uncertainty and model-shift uncertainty inside a sensitivity region Θ, together with uncertain priors, likelihoods, and dependencies;
7. preregistered decision thresholds and an abstention rule; and
8. the complete raw assurance vector (6)–(7) beside the scalar.

For fixed θ, Bayes' rule gives

$$p_\theta = \Pr_\theta(H \mid E) = \frac{\pi_\theta L_\theta(E \mid H)}{\pi_\theta L_\theta(E \mid H) + (1 - \pi_\theta) L_\theta(E \mid \neg H)}. \tag{11}$$

Report sensitivity bounds, not a selected point:

$$[p_-, p_+] = \left[\inf_{\theta\in\Theta} p_\theta,\ \sup_{\theta\in\Theta} p_\theta\right]. \tag{12}$$

If this interval crosses a preregistered decision boundary, or if any required likelihood or dependence term lacks calibration, the result is `abstain`. No arithmetic on ordinal labels may be substituted for (11)–(12). Report p_θ as a *model-conditional posterior belief*, not generic correctness confidence. Even when a scalar is admissible, it never closes a deductive or formal obligation, erases a blocked layer, or enlarges the proposition's scope.

3 Mathematical problem-solving workflow for pid-rs

3.1 Status and scope

The canonical note has two parts. The first records observations from external sources. The second defines a protocol for pid-rs. The typeset LaTeX/PDF companion adds a reader's primer and an evidence-aggregation supplement before reproducing this canonical text byte-for-byte. Those supplements explain the protocol but do not change it. A source observation is not a pid-rs result.

This note does not validate the mathematical claims in the external sources. It also does not claim that the source workflow guarantees a correct proof. The workflow is useful because it makes the claim, search, failure, and audit steps explicit.

This note is project-defined process guidance. It has no software implementation or defining method paper. It adds no PID method, estimator, theorem, benchmark result, or validation.

The central rule is:

> An AI model can propose, refine, or attack an obligation. Only retained and replayable evidence can close the obligation.

Model confidence, polished prose, and agreement among models are not evidence classes.

3.2 Compact glossary

- **PID (partial information decomposition):** a decomposition of information that several sources provide about a target into redundancy, source-specific unique information, and synergy.
- **SxPID or shared-exclusions PID:** the paper-defined PID functional implemented here; "colored PID" is not a separate functional. A dependence coloring qualifies a sampling theorem, not PID.
- $I_{\min}$**:** the Williams–Beer minimum-specific-information redundancy functional.
- **KSG:** the Kraskov–Stögbauer–Grassberger nearest-neighbour mutual-information estimator.
- **Estimand:** the exact population quantity a procedure is intended to estimate.
- **Oracle:** a reference implementation or value source used to adjudicate another route. An oracle is evidence only within its stated construction and error contract.
- **Antichain and redundancy order:** an antichain is a nonempty collection of nonempty source subsets in which no member contains another. The declared partial order indexes PID cumulatives.
- **Zeta and Möbius transforms:** inverse finite linear transforms between lattice cumulatives and atoms. A matrix identity alone does not identify the intended event semantics.
- **Dependence coloring:** a partition of sample rows into color classes such that the complete rows within each class satisfy the theorem's declared mutual-independence premise.
- **Outward-rounded interval:** endpoints rounded down and up so the exact real value is enclosed.
- **binary64:** the IEEE 754 double-precision floating-point format.
- **MPFR and Arb:** libraries for specified-rounding multiprecision arithmetic and rigorous ball arithmetic, respectively.

- **Lean and SMT:** Lean is a proof assistant with a small proof-checking kernel; satisfiability modulo theories (SMT) solvers decide or search within supported logical theories.
- **Complete separation oracle:** an algorithm proved to find a violating object whenever one exists in the declared domain.
- **Confidence limits and multiplicity control:** sampling-uncertainty bounds and procedures that control an error criterion across multiple reported hypotheses or cells.
- **Controlled versus blind holdout:** a controlled holdout freezes and separates development from evaluation; it is blind only when the named development roles cannot access sealed rows, seeds, targets, keys, or unredacted results before the first complete adjudication.
- **Digest:** unless a claim says otherwise, a digest is lowercase SHA-256 over the exact raw bytes of the named artifact. Semantically equivalent re-encodings have different digests.
- **Ecosystem projects:** Prisoma analyzes learned representations; Galadriel monitors cross-sensor consistency; Haldir concerns authorization/reference monitoring; and Crebain is a sensor-fusion visualization consumer. Their versioned requirements are tracked in `ECOSYSTEM_CAPABILITIES.md`.

3.3 Source observations

3.3.1 X thread

The thread author reports six solutions to open Erdős problems in five days. The thread does not, by itself, verify those solutions. It describes this repeated loop:

```
attempt -> failure -> diagnosis -> new approach -> proof draft -> adversarial
↪  audit -> revision
```

The author reports runs of about 6 to 32 hours for individual problems. The author also gives these search rules:

- Restate the exact problem.
- State what a complete proof or disproof must show.
- List weaker results that do not solve the problem.
- List traps and edge cases.
- Start several independent approaches.
- Keep incompatible approaches active.
- Search for counterexamples to proposed lemmas.
- Block a route that stops at an unproved statement of comparable strength.
- Challenge each candidate argument before acceptance.

These observations come from the five linked posts:

- Thread introduction

- Exact problem and audit requirements
- Independent routes and blocker rules
- Long-run work pattern
- Research loop

3.3.2 Cycle Double Cover prompt

The Cycle Double Cover prompt starts with exact definitions. It fixes the graph class, the meaning of a cycle, edge multiplicity, disconnected graphs, and the empty graph. It then states the one result that counts as complete.

The prompt also lists results that do not count. Examples include proofs for special graph classes, finite computation, a cover with the wrong edge multiplicity, and a reduction to another unproved claim. It asks for independent approach families. It delays the sharing of ideas between routes until each route has clear strengths and gaps. It requires concrete lemmas, constructions, equations, or counterexamples. It also gives a problem-specific audit list.

Project interpretation: this structure separates three questions:

1. What is the exact claim?
2. What work can help the search but cannot complete the claim?
3. What checks can falsify a candidate proof?

3.3.3 Erdős repository

The source tree was inspected at immutable commit `f2ae0edb45cbdb257e135d51ef855f64caeb348b`. At that commit, each of the six problem directories contains a problem-specific prompt and a paper. The tree also contains Lean projects for problems 486, 788, and 1038. It contains Python numerical verifiers for problems 390, 536, and 1038.

The immutable prompt files are available for 1002, 1038, 390, 486, 536, and 788. The prompts state the quantifier order and required target strength explicitly. They list non-solutions and problem-specific failure modes. Examples include:

- an exact asymptotic limit instead of an order bound;
- one fixed infinite witness instead of a different finite witness at each scale;
- a full-sequence limit instead of a selected subsequence;
- exact extrema and sharpness instead of numerical candidates;
- strict control of limit interchange, tightness, and mass escape;
- preservation of integer, graph, or measure structure during a reduction.

Project interpretation: problem 1038 shows one certificate pattern. Its numerical verifier is designed to use a double-precision scan for initial brackets. Its higher-precision and Arb interval checks are designed to certify signs. The Lean project contains rational and interval data for finite kernel checks. This review did not run the verifier or build the project. The repository also has immutable 486 and 788 Lean trees. It has numerical verifiers for 390 and 536.

Project interpretation: no complete chronological failure log was found in the inspected commit. The prompts request a route registry and blocker policy, but this inspection did not find a full transcript. This difference matters when pid-rs records its own process.

3.3.4 Erdős 477 cubic repository

The pw/erdos477-cubic repository was inspected at immutable commit 8440d599890b5a5ef7b212c65338723ab2443eaf. This was a process review only. It did not validate the repository's claimed theorem.

Five process choices are useful for pid-rs:

1. The repository pins a semantic ambiguity against the primary problem statement before using the proof. In its case, it distinguishes uniqueness of an image value from uniqueness of a parameter that enumerates that value.
2. It preserves the earlier, narrower cubic argument when the claim expands to all higher powers. The narrower route remains a worked instance and an independent audit target instead of being overwritten by the generalization.
3. Its VERIFICATION.md separates elementary bookkeeping from two imported determinant-method results. It explicitly says that two proof narratives that share the same imported crux are correlated evidence, not two independent confirmations.
4. It retains the actual adversarial briefs for the cubic claim and the generalized claim. Each brief names concrete failure targets, asks for refutation rather than confirmation, and distinguishes a surviving argument from an independently re-derived one.
5. An adversarial pass found an incorrect genus formula. A cheap invariant exposed it: the old formula produced a non-integer genus in one case. The repository corrected the formula, identified the exact dependency subgraph that used it, and recorded why the main theorem did not depend on that subgraph.

The same record also shows limits that pid-rs should not copy. The construction and adversarial passes used the same model family. The raw sessions are not retained in the repository. No formal proof or human specialist review is present. Its primary-source checks establish that cited statements exist and that an application appears to meet their hypotheses; they do not re-prove the cited determinant methods. Therefore, labels such as "full proof" in that repository are not evidence for pid-rs and do not change any pid-rs disposition.

Project interpretation: four transferable controls should be added below—critical-cut-set accounting for correlated routes, an imported-theorem application map, a load-bearing correction ledger, and cheap invariant probes. These controls improve auditability without importing the external mathematics.

3.3.5 Corrected vector-bundle claim: a typed citation-edge failure

The public correction chain beginning with the original claim, the linked draft, Tony Feng's specific objection, the author's acknowledgement, and the conspicuous correction post was inspected on 2026-07-25. The downloaded 18-page draft had SHA-256 ebb5aa2c8d1d08cd1c7692ac0526cf537c00985bf5e129ff165be128404e69ca. The three distinct images visible in the claim/correction chain were inspected at their original resolutions: a paper-introduction image, a screenshot of Theorems A and B, and a screenshot of the correction. They repeated claims or text present in the draft/thread and supplied no independent proof.

The dedicated Chrome-extension automation transport failed twice. Codex Computer Use subsequently loaded the signed-in public thread in Chrome and exposed its current visible conversation/replies tree. Public status responses, the first public replies page, the linked draft, and the primary cited paper were used as corroborating retrieval paths; reproducing the same content does not make them independent mathematical routes. The public replies page was a third-party transport and returned 20 items, including

nested responses; its pagination cursor did not advance. These access details are session observations, not a retained browser attestation. This record therefore covers the mathematical exchange and follow-ups visible through those paths, but does not claim completeness for deleted, private, later, or unreturned replies. Social agreement, criticism, or silence was not treated as mathematical evidence.

The load-bearing error is narrower and more informative than "an AI proof was wrong." The draft's Theorem 7.1 uses the exact sequence

$$\pi^{\mathbb{A}^1}_{10,5}(S^{9,5})(\mathbb{C}) \longrightarrow \pi^{\mathbb{A}^1}_{9,5}(SL_4)(\mathbb{C}) \longrightarrow \pi^{\mathbb{A}^1}_{9,5}(SL_5)(\mathbb{C}).$$

It then reads Theorem 7.2.1 of the immutable arXiv v3 source 2306.04631v3, downloaded here with SHA-256 d4bd95572c7e2c356e964407ceab26c64c37768a655b45b45feaa4ab50dd8536, whose displayed short exact sequence is

$$0 \longrightarrow K^M_{d+2-j}/24 \xrightarrow{(\nu_d)_*} \pi^{\mathbb{A}^1}_{d+j,j}(S^{2d-1,d}) \longrightarrow GW^{d-j}_{d+1-j},$$

followed by the grammatically ambiguous words "which is an isomorphism if $j \geq d-3$." The draft attached "is an isomorphism" to the left map $(\nu_d)_*$ and, at $d = j = 5$, asserted

$$\pi^{\mathbb{A}^1}_{10,5}(S^{9,5})(\mathbb{C}) \cong K^M_2(\mathbb{C})/24 = 0.$$

The preceding source discussion, together with Theorem 7.2.2(3), assigns the range-dependent surjectivity property to the rightmost map to the Grothendieck–Witt sheaf, not an isomorphism property to $(\nu_d)_*$. Theorem 7.2.1's word "isomorphism" remains grammatically and mathematically defective in the displayed formulation, so this review does not repair it by choosing whichever referent makes the draft's argument work.

There is a stronger source-based check. The proof of Theorem 7.2.1 invokes the stable 1-line calculation of Røndigs–Spitzweck–Østvær. Its immutable 1604.00365v2 artifact, downloaded here with SHA-256 bd cf6f0ef128457740c09c5fb38c1a187951b29119d5ec2b8ba0339cb7887966, states in equation (1.1) and Theorem 5.5, for the relevant fields and range, the exact sequence

$$0 \longrightarrow K^M_2(F)/24 \longrightarrow \pi_{1,0}\mathbf{1}(F) \longrightarrow F^\times/(F^\times)^2 \oplus \mathbb{Z}/2 \longrightarrow 0.$$

ABH's stable-range comparison specializes the middle sheaf evaluation at $d = j = 5$ to this stable group. Over $F = \mathbb{C}$, both outer arithmetic simplifications are exact: every Milnor symbol $\{a, b\}$ is $24\{\alpha, b\}$ after choosing $\alpha^{24} = a$, so $K^M_2(\mathbb{C})/24 = 0$; and every nonzero complex number has a square root, so $\mathbb{C}^\times/(\mathbb{C}^\times)^2 = 0$. The remaining summand is not zero. Consequently,

$$\pi^{\mathbb{A}^1}_{10,5}(S^{9,5})(\mathbb{C}) \cong \mathbb{Z}/2.$$

Equation (27) is therefore false, rather than merely unsupported. This is an evaluation of motivic homotopy sheaves on $\mathrm{Spec}(\mathbb{C})$; it is not the separate Betti or complex- realization argument later in the draft. The characteristic and field hypotheses of both cited results apply to $\mathbb{C}$.

The specialization itself supplies the smallest retained human-readable counterexample to the wrong adjacent-arrow inference:

```
0 -> 0 -> Z/2 --id--> Z/2 -> 0
```

The right nonzero map is an isomorphism, while its adjacent map from zero is not and the middle group is nonzero. The executable checker represents the same finite group as $C_2 = \mathbb{Z}/2$,

```
0 -> 0 -> C2 --id--> C2 -> 0
```

and exhausts every element, image, and kernel. The prose and executable witnesses are two representations of one countermodel, not independent evidence routes.

Dependency reach must remain scoped. Equation (27) was used to make the second arrow in (26) injective; after separately arguing that this arrow has zero image, the draft concluded its domain was zero. The corrected first group makes that inference unavailable. Conditional on retaining the draft's separate zero-image argument, exactness now gives only a surjection

$$\mathbb{Z}/2 \twoheadrightarrow \pi^{\mathbb{A}^1}_{9,5}(SL_4)(\mathbb{C}),$$

so the target group is constrained to be either 0 or $\mathbb{Z}/2$; this audit does not choose between them. The displayed proof of Theorem 7.1 therefore fails, but this calculation does not disprove Theorem 7.1. In the draft's displayed dependency chain, Theorem 7.1 feeds Corollary 8.3, Theorem 8.4, Theorem A's no-motivic-lift and nonalgebraizability route, and Corollary 1.1. Earlier $\mathbb{P}^4$/projective machinery lies outside this particular cut: that means this error does not by itself invalidate it, not that this inspection audited or salvaged it. This inspection did not prove or disprove non-algebraizability, did not prove that a motivic lift exists, and did not adjudicate the author's later claim that other machinery in the draft may survive.

Seven lenses isolate the transferable process result:

1. **Semantic:** the anaphor "which" had two type-correct-looking referents.
2. **Logical:** a property of one arrow in an exact sequence was transferred to its neighbor.
3. **Source:** checking that a cited theorem exists did not check which morphism its qualifier names.
4. **Retrieval:** a model that was suspicious before lookup reportedly accepted the step after reading the ambiguous source, so retrieval can reinforce rather than remove an error.
5. **Independence:** repeated adversarial readings by the same model family against the same prose shared the same citation edge and count as correlated evidence.
6. **Formal:** a proof assistant can check the wrong imported premise if the human correspondence layer binds the wrong arrow; the seam needs an exact typed signature, not merely more code.
7. **Sociotechnical and scope:** public expert review happened quickly but stochastically. Its success here does not make social-media silence evidence, and one broken route does not by itself invalidate every independent result in the draft.

Project interpretation: add the typed citation-edge gate below. This is PID-neutral process evidence. No vector-bundle theorem, motivic calculation, or alternative PID definition is imported into pid-rs.

3.3.6 External AI discussions

The supplied workflow reports that three linked AI discussions were retrievable on 2026-07-24. They are process examples, not evidence that their mathematical conclusions are correct.

The Jacobian discussion shows why a review must freeze signs, orderings, normalizations, and exceptional cases before it interprets a calculation. A checked local determinant does not establish a global statement. Likewise, a fixed-support continuity result does not establish a finite-sample rate or a binary64 implementation theorem.

The unsplittable-flow discussion records a false candidate counterexample. The candidate failed when the full path closure was included. For PID, the corresponding rule is to include all antichains, event unions, supported realizations, branches, permutations, and implementation paths named by the claim.

The pid-rs correctness discussion is an external review intake. Its recommendations are work items until repository evidence replays them. Its pass or fail language does not change a project claim disposition.

3.4 AI model operating protocol

3.4.1 Applicability and risk

A **major claim** is any claim that changes a public method definition, theorem, estimator result, validated-status statement, release gate, or downstream readiness decision. A **high-consequence claim** is a major claim that could materially affect scientific inference, safety monitoring, mission or authorization policy, or a silent false numerical result. Treat an uncertain classification as the higher class until the claim packet justifies a downgrade.

Major claims require the role separation, at least two genuinely independent routes, a claim packet, a counterexample route, and an adversarial audit below. High-consequence claims additionally require a dependency-disjoint route at every load-bearing shared bridge, specialist human review, and an explicit consumer no-go condition while any applicable gate is open.

3.4.2 Separate roles

Use explicit roles and deliverables for a major claim. Do not ask one undifferentiated run to set the problem, prove it, design its test, and adjudicate its own work.

Role	Required output	Invalid shortcut
Specification editor	Frozen claim packet and ambiguity table	Solving before the claim is fixed
Source checker	Immutable source and known-result map	Trusting the prompt premise
Proof route	Named subclaims and dependency graph	Confidence language as proof
Counterexample route	Exact falsifiers and boundary cases	Only random examples
Formalization route	Prose-to-formal object map	Silent weaker surrogate
Certificate route	Exact or interval certificate format	Opaque decimal oracle
Implementation route	Specification-to-code correspondence	Unit tests as refinement proof
Statistical route	Frozen calibration or holdout protocol	Development data as holdout data
Adversarial auditor	Attack log and unresolved objections	Restating the candidate proof
Integrator	Evidence matrix and scoped decision	Majority vote among models

One worker can fill more than one role only when the overlap is recorded. A final auditor must reconstruct the claim from the frozen packet and evidence. It must not rely only on the proof author's summary.

3.4.3 Independent routes

For each major claim:

1. Give at least two routes the same frozen claim packet without the other route's answer.
2. Require each route to state its starting point, assumptions, strongest result, exceptional cases, and likely failure mode.
3. Record a route memo before routes exchange results.
4. Share route artifacts and stated implications, not hidden reasoning traces.
5. Identify shared unproved lemmas, source material, generated data, and oracles.
6. Count routes that use the same unproved bridge as one route for confidence purposes.
7. Preserve every materially distinct valid proof or solution as a separately named route, even after selecting a preferred exposition. Do not overwrite a special-case proof with a later generalization or merge two mechanisms merely because they reach the same conclusion.

Prefer method diversity to model-name diversity. Examples are a coupling proof and exact enumeration, a real-analysis proof and a formal kernel, or a generator and an independently written checker. Repeated passes by the same model against the same prompt, source text, imported theorem, or proposed proof are correlated evidence, even when they occur in fresh sessions or use harder reasoning settings. They can reveal instability and generate attacks, but they do not satisfy the independent-route requirement unless their load-bearing dependencies are genuinely disjoint.

3.4.4 Critical-cut-set accounting

Count independence at the level of proof dependencies, not prose narratives. Represent obligations as an AND/OR acyclic hypergraph: every tail of an AND-hyperedge is required, while alternative incoming hyperedges are independent OR-routes. A node closes only when one accepted incoming route has all required premises closed and its implication checked. Record every minimal shared cut set: each smallest set of open or common-cause nodes that intersects every accepted route to the target.

Two derivations that use different case splits but obtain their power from the same imported theorem, unproved lemma, generated table, numerical oracle, or implementation are correlated at that cut set. They can cross-check local algebra outside the cut set, but they do not independently close it. A route summary must therefore include:

- the critical cut set;
- all other routes that share each cut-set node;
- the evidence type that closes each node;
- whether the node was re-derived, checked against a primary source, assumed, or only restated;
- one falsification attempt directed at each shared node.

Use a dependency-disjoint route, not another paraphrase, when a high-consequence claim depends on one shared bridge. Examples include an exact enumerator versus a symbolic proof, a Python integer/Fraction checker versus a Rust MPFR producer, or a formal theorem versus a numerical fixture.

3.4.5 Frozen run context

Record this context for each non-exploratory model run:

- claim ID and revision;
- `pid-rs` commit;
- paper, generated document, and formal artifact revisions;
- proof and imported-library toolchains;
- Rust compiler, `Cargo.lock`, target, and feature set when code is in scope;
- generated table and lattice digests;
- exact definitions, conventions, assumptions, and non-solutions;
- permitted evidence classes and completion checks;
- prompt text, model identity, run date, output path, and output digest.

Without this context, treat the output as exploratory. It cannot change a claim disposition.

3.4.6 Evidence labels and route memos

The following are **artifact-verification labels** for substantive model statements. They are not the accepted evidence classes or the claim dispositions defined in the acceptance rule. Use one:

- `UNVERIFIED-MODEL-OUTPUT`;
- `CHECKED-SYMBOLICALLY`;
- `CHECKED-EXACTLY`;
- `FORMALLY-CHECKED`;
- `CERTIFIED-NUMERICALLY`;
- `IMPLEMENTATION-REFINED`;
- `EMPIRICALLY-CALIBRATED`;
- `CONSUMER-QUALIFIED`; or
- `REJECTED-BY-COUNTEREXAMPLE`.

Each route memo must record:

```
Route ID:
Claim revision:
Mathematical family:
Independent starting point:
Current obligation:
Strongest established result:
Exact evidence:
Counterexamples attempted:
Exceptional cases:
Missing lemma or bridge:
State:
Reopen condition:
Artifact paths and digests:
```

A route that reduces the target to an unproved claim of comparable strength is blocked, not complete.

3.5 pid-rs protocol

The rest of this note defines a repository protocol. It is an adaptation, not a source claim.

3.5.1 1. Write an exact-claim packet

Create one packet before work starts. Give the packet a stable claim ID. Include these fields:

Field	Required content
Claim	One mathematical or statistical statement
Objects	Exact domains, alphabets, measures, lattices, and sample spaces
Quantifiers	Their full order, including every uniformity requirement
Assumptions	Support, dependence, moments, stationarity, sample size, and numerical model
Units	Natural logarithms and nats, unless the claim states another unit
Conclusion	Exact equality, inequality, limit, coverage, error rate, or abstention rule
Non-solutions	Weaker statements that do not complete the claim
Falsifiers	Boundary cases and counterexamples that can refute the claim
Evidence class	Mathematical proof, machine-checked proof, certified computation, test, or holdout result
Completion check	A mechanical or human check that can close the claim

Do not overwrite a claim packet after you inspect a result. To correct or change it, create a new revision and retain the old revision.

Add a semantic pin for every ambiguity that changes the mathematical object. Quote or transcribe the controlling primary-source statement, record the competing readings, choose one reading with a reason, and state which downstream obligations depend on it. An implementation test cannot repair a proof about the wrong object.

3.5.2 2. Build an obligation graph

Split the claim into named obligations. Each edge must state why one obligation implies its destination obligation. Use separate nodes for these classes:

- definition and semantics;
- mathematical reduction;
- finite algebra;
- analytic inequality or limit;
- numerical certificate;
- estimator behavior;
- software conformance;
- statistical validation;
- downstream acceptance.

If an edge depends on an unproved lemma, create a separate obligation node. Mark it as open until evidence closes it.

3.5.3 3. Keep an independent approach registry

Use one row for each mathematical idea. Do not use one row for each worker or prompt.

Field	Meaning
Approach ID	Stable identifier
Family	Main mathematical mechanism
Independent inputs	Ideas not copied from another route
Current obligation	The exact node under study
Best result	Strongest proved statement
Counterexample search	Cases that were tested
Missing lemma	Exact open step
State	Proposed, active, blocked, falsified, merged, or complete
Reopen condition	New fact that can justify more work
Artifact links	Notes, code, proof files, fixtures, and logs

For PID work, start with different families when they apply. Useful families include Möbius and lattice algebra, measure-theoretic analysis, concentration inequalities, information geometry, optimization, exact finite enumeration, and certified interval analysis.

Do not give every route the current preferred answer. First, let each route state its own assumptions and failure modes. Combine routes only after this step.

3.5.4 4. Retain blockers and failed routes

Record every failed route. Keep its exact statement and its failure reason. Use one of these failure classes:

- a concrete counterexample;
- a false intermediate lemma;
- a missing assumption;
- a quantifier error;
- a reduction to a claim of comparable strength;
- a numerical certificate that does not cover the full domain;
- a machine-checked proof that checks only a weaker statement;
- a statistical design that reuses development data.

Do not delete an invalidated derivation. Mark it clearly as invalid. Add a small counterexample when one is available. State why it falsifies the route. A route can reopen only when it has a new mechanism or a stronger premise that the claim packet permits.

3.5.5 Load-bearing correction ledger

When an audit finds an error, do not classify it informally as “minor” or “fatal.” Trace it through the obligation graph. Record:

Field	Required content
Error	The exact false statement, code path, or certificate field
Detector	Counterexample, invariant, proof checker, mutation, or source comparison
Smallest witness	Minimal retained input that exposes the error
Dependency reach	Every claim node reachable from the false node
Load-bearing status	Whether any permitted conclusion depends on that node
Correction	New statement or implementation and why it is valid
Regression	A test, theorem, or mutation that fails before and passes after the correction
Residual boundary	What the correction still does not establish
Artifact revisions	Old and new claim, proof, source, and evidence digests

A non-load-bearing error still remains in the ledger. “The conclusion survives” is acceptable only after the dependency reach is explicit and an independent path establishes the conclusion without the false node.

If a result expands from a special case to a uniform or higher-dimensional theorem, create a new claim revision. Preserve the narrower proof and its evidence. Re-open all obligations involving new quantifiers, uniform constants, exceptional families, or imported theorems. Agreement on the old special case does not validate the generalization.

3.5.6 5. Convert computation into certificates

Use numerical work to find structure. Do not treat a plot or a floating-point match as a proof. Use this conversion path when it applies:

```
exploratory computation
    -> exact reduction
    -> rational or outward-rounded interval data
    -> finite certificate statement
    -> machine check
    -> independent implementation replay
```

Keep the generator, its inputs, the generated certificate, the checker, and their digests. Add documented negative mutations. The checker must reject each mutation.

Rational probabilities do not imply rational logarithmic information values. Use a symbolic identity or a certified interval for a proof claim. Label an uncertified high-precision value as a reference. Do not call a decimal reference an exact entropy oracle.

3.5.7 6. Run an adversarial audit

Audit each candidate result through five review categories:

1. **Semantic review:** Does the statement preserve the estimand, object class, and quantifier order?

2. **Mathematical review:** Can a small, boundary, singular, or dependent example falsify a lemma?
3. **Formal review:** What does the proof assistant check? What remains an assumption or prose proof?
4. **Numerical review:** Can rounding, cancellation, overflow, ties, or an unstable transform change the result?
5. **Statistical review:** Does the design control sampling error, dependence, selection, and repeated testing?

An audit must try to find a counterexample, a violated premise, or a numerical failure. It must not only restate the proof. Record the challenge, result, and revision. If the revision changes an assumption, create a new claim-packet revision.

3.5.7.1 SxPID definition-compatibility firewall

PID is not one uniquely defined functional. A theorem, fixture, estimator result, atom sign, or calibration statement from another PID must not be used as evidence for shared-exclusions PID merely because both methods use the words redundancy, unique information, and synergy. Before importing a PID result, bind all of the following:

1. the exact measure and immutable definition revision;
2. the target, ordered sources, source collections, and antichain order;
3. the cumulative event semantics, informative/misinformative split, and Möbius convention;
4. the probability domain, estimator, preprocessing map, units, and numerical representation;
5. the source theorem's hypotheses, including every positivity or identity axiom; and
6. an explicit mapping theorem stating the property preserved by the transfer.

Shared atom names, the same three mutual-information coordinates, a similar benchmark value, or a common lattice are not a mapping theorem. In particular, Williams–Beer $I_{\min}$, BROJA, CCS, MMI, SID, and other authors' PID definitions are comparison objects unless this mapping is proved. Categorical SxPID and continuous shared-exclusions estimators belong to the same scientific line, but a discrete exact-real theorem still does not validate a continuous estimator or an unquantized estimand. A measure-independent theorem may be imported only after its abstract objects and all hypotheses are instantiated with the actual SxPID definitions. Record failed mappings as negative results rather than silently transplanting them.

3.5.7.2 Imported-theorem application map

For every external theorem that carries a load-bearing step, retain an application record:

```
Source theorem and immutable version:
Exact source statement:
Named source arrow (domain -> codomain):
Property and direction imported (injective/surjective/isomorphic/zero/exact):
Local variables-to-source variables map:
Required hypotheses:
Evidence for each hypothesis:
```

```
Exceptional cases and exclusions:
Uniformity and constant dependence:
Named local arrow (domain -> codomain):
Arrow-level type and direction correspondence:
Source-language ambiguities and resolution evidence:
Conclusion actually imported:
Local obligation closed:
What was checked against the source:
What was not re-proved:
```

Checking that a theorem exists is not checking its application. Checking the application is not re-proving the source theorem. State both boundaries. In particular, audit whether an implied constant is uniform over the changing family used locally, whether an exception clause is exhaustive, and whether a transformed object remains in the source theorem's domain.

3.5.7.3 Citation-edge type check

An imported statement containing more than one morphism, an exact sequence, a commutative diagram, or an anaphor such as "which," "it," or "this map" cannot close an obligation until the following gate passes:

1. Give every source morphism a stable local identifier and a typed signature, for example $f_s : A_s \to B_s$ and $g_s : B_s \to C_s$.
2. Bind every imported predicate to one named arrow and direction: for example `Mono(f_s)`, `Epi(g_s)`, `Iso(g_s)`, or `Zero(g_s)`. A free-floating "is an isomorphism" fails.
3. Name the local arrow and show the domain, codomain, variance, indexing, and direction match the source arrow under the recorded variable map.
4. Resolve ambiguous source prose from the upstream proof, adjacent lemmas, errata, or a specialist source check. If competing readings remain, retain both and mark the obligation BLOCKED; do not choose the reading that makes the local proof work.
5. Re-derive the local implication from the typed predicate without copying the source prose. For an exact sequence, explicitly use the appropriate kernel/image consequence rather than transferring injectivity, surjectivity, or isomorphism to an adjacent arrow.
6. Run an adjacent-arrow mutation: attach the predicate to each neighboring arrow in turn. The typed correspondence or a retained countermodel must reject every wrong attachment. Keep `0->0->Z/2--id-->Z/2->0` as the minimal human-readable regression for the failure exposed above.
7. Separate source retrieval from consequence checking. A model that retrieves and then audits the same ambiguous sentence is not an independent route. Include a source-blind derivation of the local consequence and a separate primary-source/proof inspection for the imported premise.
8. Bind the same arrow identifier, signature, and predicate to any Lean theorem, executable schema, or certificate field. Treat a deep unformalized source theorem as an explicit typed premise; do not imply that an axiom or interface theorem verifies its truth.

Record CLOSED only when all applicable fields pass. OPEN means evidence remains to be supplied; BLOCKED means the source correspondence is ambiguous or false. The PDF checker retains the template,

this gate, and its negative control in the human-readable artifact. That artifact gate does not inspect every future proof automatically; each claim packet must instantiate and review the record.

The deterministic `check-citation-edge-countermodel.py` gate exhausts the finite C_2 witness's map tables, images, kernels, and isomorphism predicates. The companion mutation self-test script, `check-citation-edge-countermodel-self-test.py`, requires rejection of wrong-arrow binding, witness collapse, broken exactness, stale Markdown/LaTeX parity, and removal of the typed source-arrow field. Both run in the mathematical-workflow PDF gate, its `just` route, and CI. They reject the local inference schema; they do not interpret motivic homotopy or verify a PID theorem.

3.5.7.4 Cheap invariant probes

Before a long proof or expensive certificate run, derive low-cost invariants that every candidate must satisfy. Use them as early falsifiers and retain the smallest failure. Examples include:

- an integer-valued quantity must remain integral;
- a probability vector must have nonnegative entries summing exactly to one;
- a Möbius transform and its zeta inverse must reconstruct every coordinate;
- a symmetry claim must commute with the declared source permutation;
- an interval lower endpoint must not exceed its exact partial sum, and its upper endpoint must contain a separately proved tail bound;
- a claimed covariance or concentration proxy must have the required sign and limiting behavior;
- a resource estimate must dominate the exact serialized object it is intended to bound.

An invariant can refute a route. Passing it does not prove the route. Whenever an invariant catches an error, add it to the correction ledger and convert it into a permanent negative control.

3.5.8 7. Use a blind holdout protocol

A proof and a holdout test answer different questions. A proof can establish a theorem under stated assumptions. A holdout test can estimate the behavior of an estimator or implementation on a defined benchmark distribution. Do not use one as a substitute for the other.

Call a benchmark blind only when the development and implementation roles cannot access the holdout rows, holdout seeds, target values, or unredacted adjudication output before the frozen analysis produces the complete first result table. Preregister every reported output. Record an access matrix for all roles and assets. Record who holds each sealed artifact and its immutable digest. If these conditions do not hold, call the design a controlled holdout, not a blind holdout.

Use this protocol for estimator and pipeline claims:

1. Freeze the estimand, data-generating families, parameter grid, sample sizes, metrics, tolerances, failure rules, and analysis code.
2. Seal the generator, seed list, and targets. Publish an independently timestamped digest before holdout access.
3. Define the development, implementation, execution, and adjudication roles. Record all role overlaps and the access matrix.

4. Separate development, calibration, and holdout data. Do not tune on the holdout data.
5. Keep the holdout rows, seeds, target values, and unredacted result output unavailable to the development and implementation roles until the adjudicator records the complete first result table.
6. Include positive controls, negative controls, boundary cases, and known failure regimes.
7. Use repeated holdout draws for sampling claims. State confidence limits and preregistered acceptance criteria.
8. Report every planned cell. Do not remove a cell after a poor result.
9. Report effect errors, interval coverage, abstentions, failures, and resource limits.
10. Apply the stated multiplicity control when the decision uses many cells or hypotheses.
11. Record the first holdout result before a revision. A revision needs a new holdout version.
12. Publish the generator, manifest digest, adjudicator, and full result table after the blind phase.

The benchmark must define its scope. Synthetic Gaussian performance does not prove performance for mixed, singular, quantized, or dependent laws. A fixed benchmark does not prove a general theorem.

3.5.8.1 Minimum blind-benchmark commitment

The pre-access commitment must contain enough information to detect a change after result access. Record these fields:

Field	Required content
Benchmark ID	Stable versioned identifier
Claim IDs	Scoped empirical or calibration claims that the benchmark can accept or reject; never an unspecified universal or exact theorem
Analysis-plan digest	Digest of the metrics, tolerances, confidence limits, multiplicity rule, and acceptance rule
Code identity	Immutable source identity and environment or package-lock identity
Generator identity	Generator digest, parameter-grid digest, and sampling-law version
Sealed inputs	Digest of the seed list, target bundle, or encrypted holdout package
Role separation	People or systems that develop, implement, execute, and adjudicate, including all role overlaps
Access matrix	Permitted access for each role to code, rows, seeds, targets, keys, and result output
Digest custody	Party that holds each sealed artifact and its immutable digest
Access rule	Event that permits target access and the party that records that event
Independent time evidence	Third-party timestamp or an independent adjudicator record
Failure rule	Treatment of crashes, non-finite values, abstentions, missing cells, and resource exhaustion
Result identity	Digest of the complete first result table before any revision
Deviations	Every difference from the frozen plan

A local Git commit time alone is not independent time evidence. Do not store a secret seed, target, decryption key, or credential in the repository. The adjudicator must retain failed cells and must run the frozen analysis without manual result-dependent changes. If a security constraint prevents public release of a sealed input, publish its digest and state who controls access.

This protocol reduces result-dependent tuning and selective reporting. It does not make a benchmark distribution representative, prove that an analytic target is correct, or replace a theorem.

3.5.9 8. Maintain a claim-to-evidence matrix

Use the matrix to prevent evidence substitution.

Claim class	Evidence that can support it	Evidence that does not complete it
PID definition and provenance	Defining paper, exact repository definition, and provenance marker	Similar name or similar output
Möbius reconstruction identity	Symbolic derivation and machine-checked finite algebra	Random numerical examples
Population functional theorem	Proof with explicit assumptions and counterexample audit	Passing implementation tests
Finite-alphabet consistency	Probability proof, stated mode of convergence, and formalized subclaims	One Monte Carlo curve
Numerical reference value	Symbolic value, rigorous interval, or high-precision value with error limit	Default floating-point agreement
Rust implementation conformance	Independent oracle, mutation test, feature-path parity, and boundary tests	A paper citation
Scoped estimator calibration	Declared data-generating family, repeated preregistered holdout draws, confidence limits, and acceptance criteria	Reused development fixtures or one unqualified draw
Dependence-aware inference	Valid dependence assumptions plus block or permutation design checks	An independent-row test
Downstream suitability	Consumer-specific input contract and acceptance fixture	A generic PID example
Limitation or impossibility	Explicit counterexample or proved obstruction	Failure to find a proof

Each new or revised scientific claim must identify its applicable claim class or classes, evidence, and scope. A result can use more than one claim class. For example, a correct functional identity does not establish estimator calibration.

3.6 Application to shared-exclusions PID

The primary theoretical target in this project is the shared-exclusions PID line. Keep paper-defined quantities separate from project-defined diagnostics, wrappers, and workflows. `method-catalog.js` on and its generated `METHODS.md` rendering are the authorities for method origin, code availability, and validation status.

3.6.1 Exact claim packet for a new PID theorem

The packet must define:

- the source collection and target;
- the source antichains and lattice order;
- the pointwise informative, misinformative, and net terms;
- the averaging law;

- the logarithm base and units;
- the population support and any minimum-mass condition;
- the row dependence model;
- the estimator and all fitted preprocessing;
- the requested conclusion and its convergence mode;
- the status of negative atoms;
- all cases where the method must abstain.

If the theorem concerns a plug-in estimator, distinguish the population functional from the empirical law and the implementation. If it concerns continuous data, state the support model. Do not infer full-dimensional absolute continuity from unique observed rows.

3.6.2 Independent PID approach families

For each new theorem, use at least three applicable audit families. If fewer apply, record why.

- **Combinatorial:** antichain order, Möbius inversion, atom reconstruction, and source symmetry.
- **Analytic:** continuity, support boundaries, limiting arguments, and perturbation bounds.
- **Probabilistic:** concentration, dependence colorings, mixing assumptions, and resampling.
- **Computational:** exhaustive small alphabets, exact count tables, and counterexample search.
- **Formal:** Lean or SMT statements for finite algebra and deterministic inequalities.
- **Statistical:** coverage, power, null calibration, selection effects, and blind holdout behavior.

These families can support each other. They are not interchangeable. A formal lattice identity does not prove a concentration theorem. A concentration theorem does not prove that a continuous kNN estimator targets the intended functional.

3.6.3 Required counterexample search

For each new PID claim, test the relevant cases:

- independent sources and target;
- copied sources;
- XOR and other synergy gates;
- duplicate or deterministic sources;
- zero-probability and near-zero-probability cells;
- support deletion and support creation;
- singular and mixed-dimensional continuous laws;
- exact ties and quantized observations;

- row dependence and a false dependence partition;
- cancellation in reconstructed atoms;
- source permutation and target relabeling;
- serial and parallel implementation paths.

Keep a counterexample when it breaks a proposed unconditional claim. State the corrected assumption next to the retained example.

3.6.4 Formal and software evidence

Formalize the exact theorem statement before large certificate generation. Record the proof kernel, toolchain, imported axioms, and unformalized steps. For finite domains, use exact enumeration or certified intervals when possible. Then replay the same cases through the Rust API.

Use mutation tests for proof and evidence scripts. Add documented negative mutations. The check must reject each mutation. Run every API, feature, and build mode named by the claim.

3.6.5 Statistical and ecosystem evidence

Define separate benchmark strata for categorical SxPID, fitted quantized SxPID, continuous KSG terms, composed PID atoms, and uncertainty procedures. Each stratum needs its own analytic target or certified numerical reference. If a stratum has neither, label it diagnostic. Each stratum also needs its own tolerance and abstention policy.

For Prisoma, Haldir, Galadriel, and Crebain, record a consumer contract. The contract must identify the required estimand, data support, dependence model, scale, uncertainty output, resource limit, and failure behavior. Do not claim consumer readiness until a versioned acceptance suite checks every contract field. One fixture supports only its declared case. Do not infer consumer readiness from general unit tests.

3.7 Full semantic closure

A model, solver, proof assistant, generator, or test must check the complete object in the claim packet. For a PID claim, this can include:

- every nonempty antichain for the declared source count;
- every event union and target intersection in the definition;
- every supported realization;
- every source permutation named by a symmetry claim;
- every empirical count table in a declared exhaustive domain;
- informative, misinformative, net, tie, and abstention branches;
- every API, feature, serial or parallel path, and specialized or general path named by the claim;
- every fitted transform and split named by a consumer contract.

If complete enumeration is impossible, supply a proved reduction or a complete separation oracle. Random examples do not establish a universal equivalence statement. State the exact bound of every finite search.

3.8 PID exceptional-case checklist

Boundary analysis is a separate obligation. For each applicable item, state whether the theorem extends, changes statement, or requires abstention:

- zero or near-zero supported mass;
- support deletion or creation;
- an event denominator that approaches zero;
- exact or near `I_min` ties;
- informative and misinformative cancellation or an atom near zero;
- duplicate, copied, deterministic, or constant variables;
- an invalid dependence coloring;
- a transform fitted on evaluation rows;
- drift, adaptive selection, or repeated use;
- a zero KSG radius or nonunique neighbor shell;
- unequal intrinsic dimensions or incompatible reference measures;
- a mixed-dimensional or singular law;
- integer, allocation, recursion, and cancellation boundaries;
- platform-dependent transcendental arithmetic.

A derivation that divides away a boundary must keep a separate obligation for that boundary.

3.9 Layered assurance and go/no-go gates

Use each applicable layer. No layer substitutes for another.

Gate	Required closure	Claim disposition while open	Prohibited wording while open
G0 Claim identity	Frozen packet, sources, non-solutions	`blocked`	Claim scope is final
G1 Conventions and premises	Convention and assumption map	`blocked`	Premises are discharged
G2 Mathematical core	Proof or counterexample and boundary audit	`active`	The theorem or disproof is complete
G3 Formal semantics	Actual objects and implication in a proof checker	`active` or `blocked`	Formally verified
G4 Certified numerics	Rigorous enclosure and unresolved semantics	`active` or `blocked`	Certified sign or tie
G5 Executable conformance	Refinement or bounded complete equivalence	`active` or `blocked`	Verified implementation
G6 Estimator calibration	Preregistered scoped calibration	`active` or `blocked`	Calibrated beyond the tested scope
G7 Consumer qualification	Versioned contract and acceptance suite	`blocked`	Consumer-ready or authority-grade
G8 Release archive	Reproducible builds, hashes, and first-result record	`blocked`	Final release assurance

An inapplicable gate needs a written reason. A release statement must name the closed and open layers. `falsified` is terminal only after an accepted counterexample closes a negative result; it is not an "open" G2 state.

For a major claim, keep a revision-preserving directory:

```
claims/<CLAIM-ID>/
  claim-v1.md
  conventions.md
  obligations.md
  routes.md
  failures/
  certificates/
  formal/
  implementation/
  benchmark/
  audit.md
  evidence-matrix.md
  decision.md
```

Do not add empty placeholders. Create a file when it has evidence or a required open obligation. Do not overwrite old claim revisions, failed routes, certificate inputs, or first-result records.

3.10 Acceptance rule

Record an evidence label and a separate claim disposition. Use one of these evidence labels:

- theorem proved under stated assumptions;

- machine-checked finite obligation;
- certified numerical bound;
- preregistered empirical result;
- counterexample;
- diagnostic observation; or
- no accepted evidence.

Use one of these dispositions: proposed, active, blocked, falsified, or complete. A result is complete only when all applicable obligations are closed. A closed finite or machine-checked sub-obligation does not close its parent claim. A counterexample can complete a disproof but cannot complete the original proof claim.

Contradictory accepted-looking evidence forces the claim to `blocked`. Retain both artifacts, identify the smallest obligation on which they disagree, and resolve that conflict before either route can close the claim.

Do not change an evidence label or disposition without new evidence. Link all artifacts with the claim ID. This record helps reviewers find gaps. It does not prove the claim.